Checkpoint 5 - Computational Thinking using Python

Matheus Yudi | RM571245 | 1TDSPF

Erick Menezes | RM570325 | 1TDSPF

- 1. Visão Geral da Solução

Cenário

Hoje em dia, muitos sistemas precisam buscar dados de fora, tratar essas informações e guardar tudo num banco de dados. Foi isso que fizemos neste projeto, usando previsão do tempo. Nosso programa consulta a previsão dos próximos 7 dias de quatro cidades (São Paulo, Rio de Janeiro, Curitiba e Manaus), faz alguns cálculos com esses dados e grava o resultado numa tabela do Oracle da FIAP. Com tudo no banco, dá para consultar e comparar as cidades depois usando SQL.

Como funciona

1. O programa pede a previsão de cada cidade para a API Open-Meteo, usando a biblioteca requests.
2. Com os dados que voltam (em JSON), ele calcula a temperatura média e a amplitude térmica de cada dia, e classifica o dia e a chuva.
3. Depois, conecta ao Oracle com a biblioteca oracledb, cria a tabela previsao_tempo (se ela ainda não existir) e grava os registros. Se tudo der certo, usa commit(). Se der algum erro, usa rollback(), para não ficar nada gravado pela metade.

Por que escolhemos a Open-Meteo

- É gratuita para uso não comercial e não precisa de chave nem de cadastro, então atende ao requisito de API sem autenticação complexa.
- Não está entre as APIs proibidas no enunciado (ViaCEP e PokeAPI).
- Traz dados numéricos (temperaturas e chuva), o que permite fazer cálculos e classificações, e não só copiar texto para o banco.
- O JSON vem bem organizado e é fácil de ler no Python.
- Como as cidades têm climas bem diferentes, as classificações variam. Nos nossos testes, Curitiba apareceu como Frio em um dia e Manaus como Quente em todos os dias, o que mostra que as regras funcionam.

- 2. Especificação da API Consumida

API utilizada: Open-Meteo (previsão do tempo)

URL base: https://api.open-meteo.com/v1/forecast

Sobre a API

A Open-Meteo é uma API pública de dados do tempo. Ela é gratuita para uso não comercial e não precisa de chave nem de cadastro, o que atende à exigência do Checkpoint de usar uma API sem autenticação complexa. No programa, usamos o método GET com a biblioteca requests, e a resposta chega em JSON.

Parâmetros enviados na requisição:

Parâmetro	Valor usado	Função
latitude	ex.: -23.55	Latitude da cidade consultada
longitude	ex.: -46.63	Longitude da cidade consultada
daily	temperature_2m_max,temperature_2m_min,precipitation_sum	Dados diários pedidos: temperatura máxima, mínima e chuva total
timezone	America/Sao_Paulo	Fuso horário das datas retornadas
forecast_days	7	Quantidade de dias de previsão

Cidades consultadas

São Paulo, Rio de Janeiro, Curitiba e Manaus. O programa faz uma requisição por cidade, e cada uma traz 7 dias de previsão. Ou seja, uma execução completa rende 4 cidades × 7 dias = 28 registros.

Como é a resposta

Os dados de cada dia ficam dentro do campo daily, em listas separadas. A posição 0 de todas as listas é o mesmo dia, a posição 1 é o dia seguinte, e assim por diante. Exemplo real da consulta de São Paulo:

```
"daily": {
   "time": ["2026-10-29", "2026-10-30", ...],
   "temperature_2m_max": [19.8, 23.7, ...],
   "temperature_2m_min": [16.3, 16.6, ...],
   "precipitation_sum": [14.5, 15.2, ...]
}
```

As unidades vêm no campo daily_units: °C para temperatura e mm para chuva.

Tratamento de erros

O programa olha o status da resposta e também trata falhas de conexão:

- **200:** deu certo, os dados são lidos normalmente.
- **404:** mostra a mensagem de endereço não encontrado.
- **500 ou mais:** mostra a mensagem de falha no servidor da API.
- **Outros status:** mostra o código que a API devolveu.
- **Sem conexão ou demora demais (mais de 10 segundos):** mostra a mensagem de erro de conexão.

Quando acontece algum erro, o programa pula aquela cidade e continua com as outras.

- 3. Regras de Negócio / Transformações Aplicadas

Depois de consultar a API, o programa recebe, para cada cidade, listas separadas com as datas, as temperaturas máximas e mínimas e a chuva prevista de cada dia.

Usamos o zip() para juntar essas listas, de modo que cada dia vire um registro completo. Depois disso, cada registro passa por quatro transformações antes de ir para o banco.

Transformação 1: temperatura média (cálculo)

Somamos a temperatura máxima com a mínima, dividimos por 2 e arredondamos para uma casa decimal. Exemplo: máxima de 19,8 °C e mínima de 16,3 °C dão média de 18,1 °C.

Transformação 2: amplitude térmica (cálculo)

É a máxima menos a mínima, também com uma casa decimal. Mostra o quanto a temperatura variou durante o dia. No mesmo exemplo: 19,8 − 16,3 = 3,5 °C.

Transformação 3: classificação do dia (categorização)

Classificamos o dia de acordo com a temperatura média:

Média	Classificação
Abaixo de 18 °C ---->	Frio
**De 18 °C até menos de 24 °C ---> **	Ameno
24 °C ou mais ---->	Quente

Transformação 4: Nível de chuva (categorização). A chuva prevista, em milímetros, é classificada assim:

Chuva prevista	Nível
0 mm ------>	Sem chuva
Até 5 mm ------>	Fraca
**Mais de 5 mm até 15 mm ---> **	Moderada
Mais de 15 mm ------>	Forte

Os limites de temperatura e de chuva foram escolhidos por nós como critério do projeto. Eles não vêm da API e dá para mudar no código.

Ordem de execução: consulta à API → junção das listas com zip() → cálculo da média e da amplitude → classificação da temperatura e da chuva → gravação no Oracle.

- 4. Dicionário de Dados & Script DDL (Oracle)

Coluna	Tipo	Descrição	Restrições
id_previsao	NUMBER	Identificador automático do registro	PRIMARY KEY, gerado automaticamente (IDENTITY)
cidade	VARCHAR2(50)	Nome da cidade consultada	NOT NULL
data_previsao	DATE	Dia a que a previsão se refere	NOT NULL
temp_maxima	NUMBER(4,1)	Temperatura máxima prevista, em °C (dado da API)	NOT NULL
temp_minima	NUMBER(4,1)	Temperatura mínima prevista, em °C (dado da API)	NOT NULL
temp_media	NUMBER(4,1)	Média entre máxima e mínima, calculada pelo programa	NOT NULL
amplitude	NUMBER(4,1)	Máxima menos mínima, calculada pelo programa	NOT NULL
classificacao	VARCHAR2(10)	Frio (média abaixo de 18), Ameno (18 a menos de 24) ou Quente (24 ou mais)	NOT NULL
chuva_mm	NUMBER(5,1)	Chuva total prevista para o dia, em mm (dado da API)	NOT NULL
nivel_chuva	VARCHAR2(15)	Sem chuva (0), Fraca (até 5), Moderada (até 15) ou Forte (acima de 15)	NOT NULL

```
CREATE TABLE previsao_tempo (
id_previsao NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 cidade VARCHAR2(50) NOT NULL,
```

```
data_previsao DATE NOT NULL,
temp_maxima NUMBER(4,1) NOT NULL,
 temp_minima NUMBER(4,1) NOT NULL,
 temp_media NUMBER(4,1) NOT NULL,
 amplitude NUMBER(4,1) NOT NULL,
classificacao VARCHAR2(10) NOT NULL,
 chuva_mm NUMBER(5,1) NOT NULL,
 nivel_chuva VARCHAR2(15) NOT NULL,
 CONSTRAINT uk_cidade_data UNIQUE (cidade, data_previsao) );
```

- 5. Instruções de Execução

O que você precisa ter

- Python 3 instalado (testamos com o Python 3.14)
- Internet, para consultar a API
- Usuário e senha do Oracle da FIAP

Instalando as bibliotecas

Rode este comando no terminal:

```
pip install requests oracledb
```

Passo a passo

1. Baixe o arquivo checkpoint5.py e coloque numa pasta do computador.
2. Abra o terminal dentro dessa pasta.
3. Rode o programa:

```
python checkpoint5.py
```

4. O programa consulta a previsão das 4 cidades e mostra o total de registros (28 quando tudo funciona).
5. Quando ele pedir, digite o usuário e a senha do Oracle da FIAP. A senha não aparece na tela enquanto você digita, e isso é normal.
6. O programa conecta no banco, cria a tabela previsao_tempo (se ela ainda não existir) e grava os registros.

O que aparece na primeira execução

```
Consultando São Paulo...
Consultando Rio de Janeiro...
Consultando Curitiba...
Consultando Manaus...

Total de registros: 28
[SUCESSO]: Conectado ao Oracle!
[SUCESSO]: Tabela criada!
[SUCESSO]: 28 registros gravados, 0 já existiam.
```

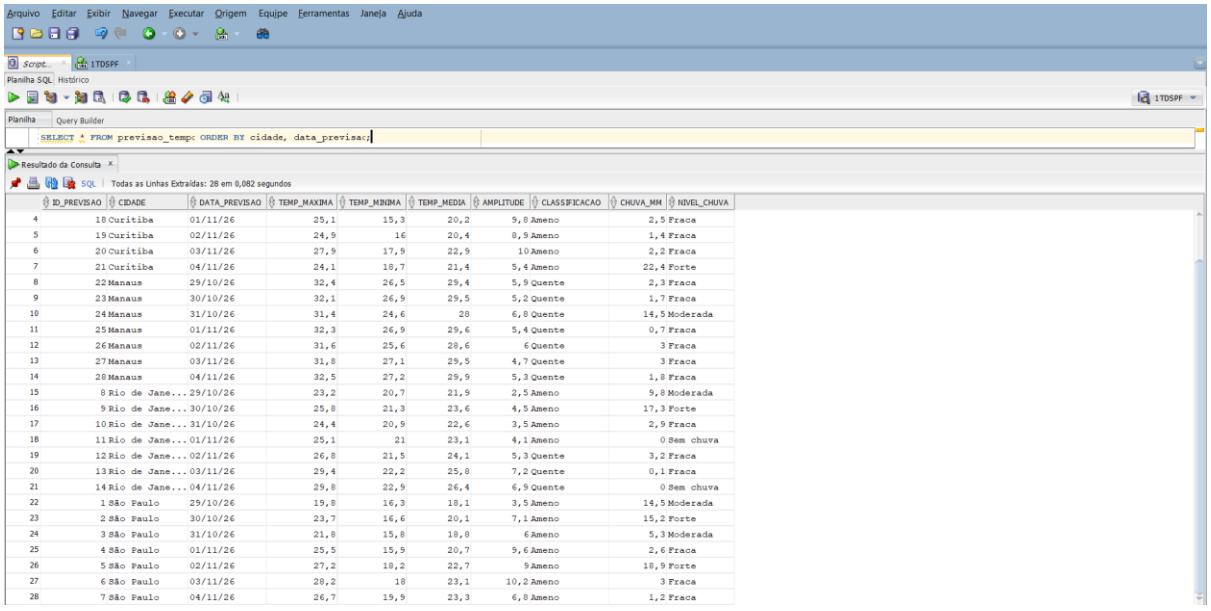
Nas execuções seguintes

A tabela já existe, então o programa só avisa e segue em frente. Os dias que já estavam gravados são ignorados, sem duplicar nada, e os dias novos da previsão são gravados. Por exemplo, ao rodar de novo no dia seguinte, aparece algo como 4 registros gravados, 24 já existiam.

Conferindo no banco

No SQL Developer, rode:

sql
```
SELECT * FROM previsao_tempo ORDER BY cidade, data_previsao;
```



ID_PREVISAO	CIDADE	DATA_PREVISAO	TEMP_MAXIMA	TEMP_MINIMA	TEMP_MEDIA	AMPLITUDE	CLASSIFICACAO	CHUVA_MM	NIVEL_CHUVA
4	18 Curitiba	01/11/26	25,1	15,3	20,2	9,8	Ameno	2,5	Fraca
5	19 Curitiba	02/11/26	24,9	16	20,4	8,9	Ameno	1,4	Fraca
6	20 Curitiba	03/11/26	27,9	17,9	22,9	10	Ameno	2,2	Fraca
7	21 Curitiba	04/11/26	24,1	18,7	21,4	5,4	Ameno	22,4	Forte
8	22 Manaus	29/10/26	32,4	26,5	29,4	5,9	Quente	2,3	Fraca
9	23 Manaus	30/10/26	32,1	26,9	29,5	5,2	Quente	1,7	Fraca
10	24 Manaus	31/10/26	31,4	24,6	28	6,8	Quente	14,5	Moderada
11	25 Manaus	01/11/26	32,3	26,9	29,6	5,4	Quente	0,7	Fraca
12	26 Manaus	02/11/26	31,6	25,6	28,6	6	Quente	3	Fraca
13	27 Manaus	03/11/26	31,8	27,1	29,5	4,7	Quente	3	Fraca
14	28 Manaus	04/11/26	32,5	27,2	29,9	5,3	Quente	1,8	Fraca
15	8 Rio de Jane...	29/10/26	23,2	20,7	21,9	2,5	Ameno	9,8	Moderada
16	9 Rio de Jane...	30/10/26	25,8	21,3	23,6	4,5	Ameno	17,3	Forte
17	10 Rio de Jane...	31/10/26	24,4	20,9	22,6	3,5	Ameno	2,9	Fraca
18	11 Rio de Jane...	01/11/26	25,1	21	23,1	4,1	Ameno	0	Sem chuva
19	12 Rio de Jane...	02/11/26	26,8	21,5	24,1	5,3	Quente	3,2	Fraca
20	13 Rio de Jane...	03/11/26	29,4	22,2	25,8	7,2	Quente	0,1	Fraca
21	14 Rio de Jane...	04/11/26	29,8	22,9	26,4	6,9	Quente	0	Sem chuva
22	1 São Paulo	29/10/26	19,8	16,3	18,1	3,5	Ameno	14,5	Moderada
23	2 São Paulo	30/10/26	23,7	16,6	20,1	7,1	Ameno	15,2	Forte
24	3 São Paulo	31/10/26	21,8	15,8	18,8	6	Ameno	5,3	Moderada
25	4 São Paulo	01/11/26	25,5	15,9	20,7	9,6	Ameno	2,6	Fraca
26	5 São Paulo	02/11/26	27,2	18,2	22,7	9	Ameno	18,9	Forte
27	6 São Paulo	03/11/26	28,2	18	23,1	10,2	Ameno	3	Fraca
28	7 São Paulo	04/11/26	26,7	19,9	23,3	6,8	Ameno	1,2	Fraca

Mensagens de erro possíveis

- [ERRO]: Falha na conexão com a API!: sem internet ou API fora do ar.
- [ERRO]: Não foi possível conectar: usuário ou senha do Oracle errados, ou o banco da FIAP inacessível.